

Before Change Demo: Boykov-Kolmogorov Refactoring Baseline

1. Purpose

This document records the behavior of the unchanged JGraphT code before refactoring the selected methods:

- BoykovKolmogorovMFImpl.grow()
- BoykovKolmogorovMFImpl.adopt()

The purpose is to create a baseline for Phase 3. After refactoring, the same tests should still pass with the same expected maximum-flow results.

No production code was changed before this baseline run.

2. Target Code

Production file:

CODE

jgrapht-core/src/main/java/org/jgrapht/alg/flow/BoykovKolmogorovMFImpl.java

Selected private methods:

CODE

grow() - line 265
adopt() - line 465

Because both methods are private, they are tested through the public maximum-flow API using the existing JUnit test class:

CODE

jgrapht-core/src/test/java/org/jgrapht/alg/flow/BoykovKolmogorovMFImplTest.java

3. Test Input

3.1 Test Command

The unchanged code was tested with:

INPUT

mvn -pl jgrapht-core "-Dtest=org.jgrapht.alg.flow.BoykovKolmogorovMFImplTest" test

3.2 Test Harness

The test class creates directed and undirected weighted graphs from edge arrays, runs BoykovKolmogorovMFImpl, and checks the computed maximum-flow value.

INPUT

```
private Graph<Integer, DefaultWeightedEdge> constructDirected(int[][] edges)
{
    return constructGraph(new DefaultDirectedWeightedGraph<>(DefaultWeightedEdge.class), edges);
}

private Graph<Integer, DefaultWeightedEdge> constructUndirected(int[][] edges)
{
    return constructGraph(
        new DefaultUndirectedWeightedGraph<>(DefaultWeightedEdge.class), edges);
}

private Graph<Integer, DefaultWeightedEdge> constructGraph(
    Graph<Integer, DefaultWeightedEdge> graph, int[][] edges)
{
    for (int[] edge : edges) {
        Graphs.addEdgeWithVertices(graph, edge[0], edge[1], edge[2]);
    }
    return graph;
}

private void testOnGraph(
    Graph<Integer, DefaultWeightedEdge> graph, int source, int sink, int expectedFlow)
{
    MaximumFlowAlgorithm<Integer, DefaultWeightedEdge> flow = createSolver(graph);
    double maxFlow = flow.getMaximumFlowValue(source, sink);
    assertEquals(expectedFlow, maxFlow, EPS);
}

private void testDirectedUndirected(
    int[][] edges, int source, int sink, int expectedDirectedFlow, int expectedUndirectedFlow)
{
    Graph<Integer, DefaultWeightedEdge> directed = constructDirected(edges);
    Graph<Integer, DefaultWeightedEdge> undirected = constructUndirected(edges);

    testOnGraph(directed, source, sink, expectedDirectedFlow);
    testOnGraph(undirected, source, sink, expectedUndirectedFlow);
}
```

The solver under test is created through the normal public API:

INPUT

```
@Override
MaximumFlowAlgorithm<Integer, DefaultWeightedEdge> createSolver(
    Graph<Integer, DefaultWeightedEdge> network)
{
    return new BoykovKolmogorovMFImpl<>(network);
}
```

3.3 Representative Input Case

The full test class contains 31 tests. A representative input from test1() is:

INPUT

```
int[][] edges = {
    { 1, 5, 10 }, { 5, 9, 20 }, { 9, 8, 35 }, { 8, 3, 27 }, { 3, 2, 49 },
    { 2, 4, 10 }, { 4, 6, 26 }, { 6, 7, 28 }, { 2, 10, 34 }, { 1, 7, 15 },
    { 1, 10, 9 }, { 3, 7, 26 }, { 3, 4, 25 }, { 3, 9, 25 }, { 3, 8, 18 },
    { 4, 9, 28 }, { 4, 3, 4 }, { 4, 7, 22 }, { 4, 8, 1 }, { 6, 2, 42 },
    { 6, 4, 13 }, { 6, 8, 43 }, { 6, 5, 48 }, { 8, 9, 8 }, { 8, 5, 34 },
    { 9, 4, 36 }, { 9, 3, 47 }, { 3, 10, 10 }, { 4, 10, 39 }, { 5, 10, 16 },
};
testDirectedUndirected(edges, 1, 10, 19, 34);
```

Expected result for this representative case:

INPUT

```
Directed graph maximum flow: 19
Undirected graph maximum flow: 34
```

4. Test Output

4.1 Summary Output

The unchanged code passed the selected test class:

OUTPUT

```
Tests run: 31
Failures: 0
Errors: 0
Skipped: 0
Build result: BUILD SUCCESS
```

4.2 Full Maven Output

The complete command output was:

OUTPUT

```
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.jgrapht:jgrapht-core >-----
[INFO] Building JGraphT - Core 1.6.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ jgrapht-core ---
[INFO] skip non existing resourceDirectory E:\avra\2-2\dp\opensource\jgrapht\jgrapht-core\src\main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ jgrapht-core ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- bundle:6.0.2:manifest (default) @ jgrapht-core ---
[INFO] Writing manifest: E:\avra\2-2\dp\opensource\jgrapht\jgrapht-core\target\classes\META-INF\MANIFEST.MF
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ jgrapht-core ---
[INFO] Copying 2 resources from src\test\resources to target\test-classes
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ jgrapht-core ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.5.6:test (default-test) @ jgrapht-core ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running org.jgrapht.alg.flow.BoykovKolmogorovMFImplTest
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.699 s -- in org.jgrapht.alg.flow.BoykovKolmogorovMFImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 15.904 s
[INFO] Finished at: 2026-06-18T23:29:29+06:00
[INFO] -----
```

5. Before-Change Code Snippets

5.1 Current `grow()` Shape

The `grow()` method processes source-tree and sink-tree active vertices in separate branches. This is one of the reasons it was selected as a companion refactoring target.

CODE

```

private AnnotatedFlowEdge grow()
{
    for (VertexExtension activeVertex = nextActiveVertex(); activeVertex != null;
        activeVertex = nextActiveVertex())
    {
        if (activeVertex.isSourceTreeVertex()) {
            // processing source tree vertex
            for (AnnotatedFlowEdge edge : activeVertex.getOutgoing()) {

                if (edge.hasCapacity()) {
                    VertexExtension target = edge.getTarget();

                    if (target.isSinkTreeVertex()) {
                        // found a bounding edge
                        if (DEBUG) {
                            System.out.printf("Bounding edge = %s\n\n", edge);
                        }

                        return edge;
                    } else if (target.isFreeVertex()) {
                        // found a node which can be added to the source tree
                        if (DEBUG) {
                            System.out.printf(
                                "Growing source tree: %s -> %s\n\n", edge, target.prototype);
                        }

                        target.parentEdge = edge;
                        target.treeStatus = VertexTreeStatus.SOURCE_TREE_VERTEX;
                        target.distance = activeVertex.distance + 1;
                        target.timestamp = activeVertex.timestamp;
                        makeActive(target);
                    } else {
                        assert target.isSourceTreeVertex();
                        if (isCloserToTerminal(activeVertex, target)) {
                            target.parentEdge = edge;
                            target.distance = activeVertex.distance + 1;
                            target.timestamp = activeVertex.timestamp;
                        }
                    }
                }
            }
        }
        } else {
            assert activeVertex.isSinkTreeVertex();

            // the logic for processing sink tree vertices is symmetrical
            for (AnnotatedFlowEdge edge : activeVertex.getOutgoing()) {

                if (edge.hasCapacity()) {
                    VertexExtension source = edge.getSource();

                    if (source.isSourceTreeVertex()) {
                        if (DEBUG) {
                            System.out.printf("Bounding edge = %s\n\n", edge);
                        }

                        return edge;
                    } else if (source.isFreeVertex()) {
                        if (DEBUG) {
                            System.out.printf(
                                "Growing sink tree: %s -> %s\n\n", source.prototype, edge);
                        }

                        source.parentEdge = edge;
                        source.treeStatus = VertexTreeStatus.SINK_TREE_VERTEX;
                        source.distance = activeVertex.distance + 1;
                        source.timestamp = activeVertex.timestamp;
                        makeActive(source);
                    } else {
                        assert source.isSinkTreeVertex();

                        if (isCloserToTerminal(activeVertex, source)) {
                            source.parentEdge = edge;
                            source.distance = activeVertex.distance + 1;
                            source.timestamp = activeVertex.timestamp;
                        }
                    }
                }
            }
        }
    }

    finishVertex(activeVertex);
}

```

```
    }  
    return null;  
}
```

5.2 Current `adopt()` Shape

The `adopt ( )` method contains the strongest code smell. It combines orphan selection, source-tree adoption, sink-tree adoption, parent search, freeing vertices, neighbor activation, and child-orphan creation.

CODE

```

private void adopt()
{
    while (!orphans.isEmpty() || !childOrphans.isEmpty()) {
        VertexExtension currentVertex;

        // child orphans take precedence
        if (childOrphans.isEmpty()) {
            currentVertex = orphans.get(orphans.size() - 1);
            orphans.remove(orphans.size() - 1);
        } else {
            currentVertex = childOrphans.removeLast();
        }

        if (currentVertex.isSourceTreeVertex()) {

            AnnotatedFlowEdge newParentEdge = null;
            int minDistance = Integer.MAX_VALUE;

            for (AnnotatedFlowEdge edge : currentVertex.getOutgoing()) {
                if (edge.getInverse().hasCapacity()) {
                    VertexExtension targetNode = edge.getTarget();

                    if (targetNode.isSourceTreeVertex()
                        && hasConnectionToTerminal(targetNode))
                    {
                        if (targetNode.distance < minDistance) {
                            minDistance = targetNode.distance;
                            newParentEdge = edge.getInverse();
                        }
                    }
                }
            }

            if (newParentEdge == null) {
                currentVertex.timestamp = FREE_NODE_TIMESTAMP;
                currentVertex.treeStatus = VertexTreeStatus.FREE_VERTEX;

                for (AnnotatedFlowEdge edge : currentVertex.getOutgoing()) {
                    VertexExtension targetVertex = edge.getTarget();
                    if (targetVertex.isSourceTreeVertex()) {
                        if (edge.getInverse().hasCapacity()) {
                            makeActive(targetVertex);
                        }
                        if (targetVertex.parentEdge == edge) {
                            targetVertex.makeOrphan();
                            childOrphans.addFirst(targetVertex);
                        }
                    }
                }
            }
            else {
                makeCheckedInThisIteration(currentVertex);
                currentVertex.parentEdge = newParentEdge;
                currentVertex.distance = minDistance + 1;
            }
        }
        else {
            assert currentVertex.isSinkTreeVertex();

            AnnotatedFlowEdge newParentEdge = null;
            int minDistance = Integer.MAX_VALUE;
            for (AnnotatedFlowEdge edge : currentVertex.getOutgoing()) {
                if (edge.hasCapacity()) {
                    VertexExtension targetNode = edge.getTarget();

                    if (targetNode.isSinkTreeVertex() && hasConnectionToTerminal(targetNode)) {
                        if (targetNode.distance < minDistance) {
                            minDistance = targetNode.distance;
                            newParentEdge = edge;
                        }
                    }
                }
            }

            if (newParentEdge == null) {
                currentVertex.timestamp = FREE_NODE_TIMESTAMP;
                currentVertex.treeStatus = VertexTreeStatus.FREE_VERTEX;

                for (AnnotatedFlowEdge edge : currentVertex.getOutgoing()) {
                    VertexExtension targetVertex = edge.getTarget();
                    if (targetVertex.isSinkTreeVertex()) {
                        if (edge.hasCapacity()) {
                            makeActive(targetVertex);
                        }
                    }
                }
            }
        }
    }
}

```

6. Baseline Conclusion

This output will be used as the comparison point after refactoring.